

Practical 4

Fractional Knapsack — Design and Analysis of Algorithms

Program (C)

```
// Aim: implement Fractional Knapsack problem
#include <stdio.h>

struct Item { int weight, profit; float ratio; };

int main() {
    struct Item item[] = {{10, 60}, {20, 100}, {30, 120}};
    int n = 3, capacity = 50;
    float totalProfit = 0.0;

    for (int i = 0; i < n; i++)
        item[i].ratio = (float)item[i].profit / item[i].weight;

    // sort by ratio descending
    for (int i = 0; i < n - 1; i++)
        for (int j = i + 1; j < n; j++)
            if (item[i].ratio < item[j].ratio) {
                struct Item t = item[i]; item[i] = item[j]; item[j] = t;
            }

    printf("Items (weight, profit): (10,60) (20,100) (30,120)\n");
    printf("Knapsack capacity: %d\n\n", capacity);

    for (int i = 0; i < n; i++) {
        if (capacity >= item[i].weight) {
            capacity -= item[i].weight;
            totalProfit += item[i].profit;
            printf("Take full item (w=%d, p=%d)\n", item[i].weight, item[i].profit);
        } else {
            float fraction = (float)capacity / item[i].weight;
            totalProfit += item[i].profit * fraction;
            printf("Take %.2f fraction of item (w=%d, p=%d)\n", fraction, item[i].weight, item[i].profit);
            capacity = 0;
            break;
        }
    }

    printf("\nMaximum profit = %.2f\n", totalProfit);
    return 0;
}
```

Output

```
Items (weight, profit): (10,60) (20,100) (30,120)
Knapsack capacity: 50

Take full item (w=10, p=60)
Take full item (w=20, p=100)
Take 0.67 fraction of item (w=30, p=120)

Maximum profit = 240.00
```